Département des enseignements généraux

Projet : Application client-serveur pour la gestion des livraisons

Travail pratique 1 : réalisation du module de gestion des livraisons (Parties 1 & 2)

Cours : Programmation orientée objet – INF111
Groupes : Tous
Session : Hiver 2026
Enseignants : Stéphane Duval et El Hachemi Alikacem

Date : 18 février 2026

Introduction

L'objectif de ce projet est de mettre en application les concepts orientés objet et les notions de programmation Java vus en cours. Il consiste en la réalisation d'une application client-serveur de gestion des livraisons pour une entreprise de livraison de colis. La partie serveur centralise la gestion des livraisons, tandis que la partie client implémente les opérations du livreur.

Afin de vous concentrer sur l'objectif principal du projet, vous disposerez du noyau de la partie serveur, qui gère les communications avec le client, ainsi que de la partie client complète. Votre travail consistera donc à compléter la partie serveur, que vous réaliserez en deux travaux pratiques :

1. Travail pratique 1 (TP1) : compléter la partie serveur en implémentant le module de gestion des livraisons en mettant en pratique les concepts orientés objet. L'utilisation des applications client et serveur se fera en mode console.
2. Travail pratique 2 (TP2) : implémenter l'interface graphique présentant les activités de livraison.

Ce travail vous permettra également d'avoir un aperçu de :

- La programmation réseau avec les sockets, où un serveur communique avec des clients ;
- La programmation multithread en Java.

Directives

Il est primordial de suivre les directives suivantes :

- Les noms des classes, des méthodes et des attributs doivent être ceux spécifiés dans l'énoncé.
- Tous les attributs doivent être privés.
- Les seuls littéraux présents dans le code doivent être définis comme des constantes.
- Le code doit être correctement indenté.

Gestion des livraisons

Pour son activité de livraison de colis, l'entreprise collabore avec des livreurs utilisant trois moyens de déplacement : vélo, voiture et camion. Chaque livreur possède un numéro qui l'identifie (id) et son nom. Il effectue un certain nombre de livraisons selon le moyen de déplacement dont il dispose : il s'agit de sa capacité de livraison.

Une livraison possède un identifiant numérique unique (id) et peut être régulière ou urgente. Les livraisons sont regroupées en lots numérotés séquentiellement. Ainsi, les livraisons du lot 1 sont attribuées aux livreurs, puis celles du lot 2, et ainsi de suite. En cas d'échec d'une livraison, celle-ci peut être tentée de nouveau jusqu'à trois fois.

Une livraison peut être dans l'un des quatre états suivants, il s'agit du statut de la livraison :

- En attente : indique que la livraison n'a pas encore été attribuée à un livreur
- En cours : indique que la livraison a été attribuée à un livreur
- Livrée : indique que la livraison a été effectuée avec succès
- Échouée : indique que la livraison a échouée (par exemples pour mauvaise adresse, destinataire absent, etc.)

Au départ de l'application, le serveur dispose d'un nombre de livraisons stockées dans l'ordre selon lequel elles doivent être réalisées. Il s'agit de la liste des livraisons à effectuer. Ainsi, lorsqu'un livreur se connecte et demande de recevoir des livraisons, le serveur lui envoie un nombre de livraisons selon sa capacité. Sachant que la capacité de livraison dépend du moyen de transport utilisé par le livreur : vélo, voiture ou camion.

Note : lorsqu'une une livraison est attribuée pour la première fois à un livreur, il s'agit là de la première tentative.

Une fois que le livreur reçoit ces livraisons, il communique par la suite, avec le serveur pour lui indiquer pour chaque Livraison si elle a été effectuée avec succès ou si elle a échouée.

Les règles suivantes régissent l'activité de livraison :

- Le serveur envoie à un livreur une liste de livraisons, puis les retire de la liste des livraisons à effectuer.
- Le client indique que la livraison a été effectuée avec succès. Le serveur va

placer cette livraison dans la liste des livraisons effectuées par le livreur.

- Si le client indique que la livraison a échouée alors, le serveur va la remettre dans la liste des livraisons à effectuer, en indiquant qu'il s'agit d'une nouvelle tentative. Si la livraison a échoué pour la troisième fois (on fait trois tentatives au maximum), elle est placée dans la liste des livraisons qui ont échoué.

C'est la partie client de l'application qui implémente l'activité du livreur, cela consiste à envoyer des commandes aux serveur et recevoir ses réponses. On peut lancer plusieurs instances de l'application client, correspondant à autant de livreurs.

Présentation du code fourni

Pour la réalisation de ce travail, vous disposez d'un code de démarrage composé de 2 applications :

- Le serveur (projet miniServeur). Les tâches principales du serveur sont la gestion des livraisons et l'interaction avec les livreurs.
- Le client (projet miniClient). Cette application permet aux livreurs de se connectés au serveur puis d'échanger avec lui en envoyant des commandes.

LE SERVEUR

Au lancement, le serveur commence par lire le fichier livreurs.txt contenant les livreurs enregistrés. Ensuite, il entre dans une phase d'initialisation qui consiste principalement à lancer un thread qui se met à l'attente de connexions de la part des clients et un thread qui réceptionne les commandes envoyées par les clients connectés.

Le rôle du premier thread est de d'enregistrer les clients qui se connectent.

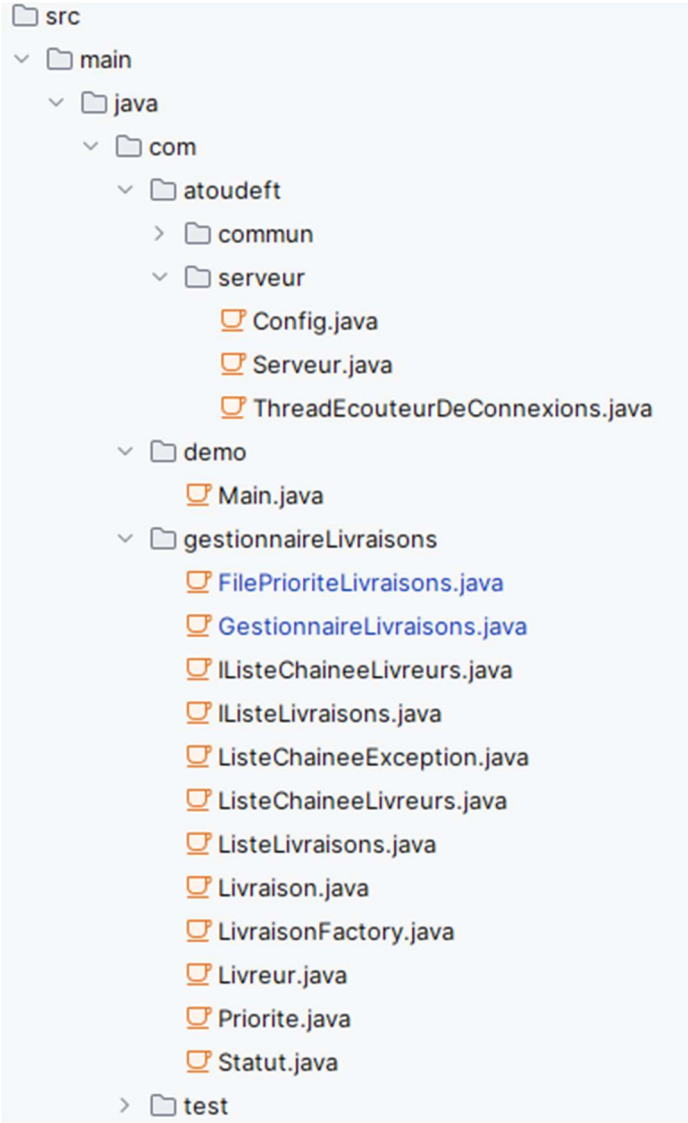
Le rôle du deuxième thread fait la réception des commandes envoyées par les clients connectés et les traite. Le traitement d'une commande revient principalement à faire la gestion des livraisons : attribuer des livraisons au livreurs, collecter les livraisons effectuées, traiter les échecs des livraisons, etc. Le traitement des commandes est délégué à la méthode nommée *traiter(Evenement evenement)* de la classe principale de gestion des livraisons : **GestionnaireLivraisons**

Voici une partie du code de la méthode *traiter(Evenement evenement) :*

```
@Override
public void traiter(Evenement evenement) {
   Object source = evenement.getSource();

   if (source instanceof Connexion) {
      Connexion cnx = (Connexion) source;
      System.out.println("GEST. LIV. a reçu : " + evenement.getType() + " " + evenement.getArgument());

      String reponse = "";
      switch (evenement.getType().toUpperCase()) {
         case "EXIT": // Le client se déconnecte.
            reponse = this.traiterEXIT(evenement);
            break;
         case "REGISTER": // Le client s'enregistre comme nouveau livreur.
            ...

      default: // La commande envoyée par le client n'a pas été reconnue.
         this.traiterCOMMAND_ERROR();
   }

   cnx.envoyer(reponse);
}
```

Pour chaque type d'événement (commande du client), la méthode *traiter()* invoque la méthode correspondante qui le traitera. À la fin, elle envoie au client la réponse qui résulte du traitement de la commande.

La figure suivante montre la structure du code du serveur :



Le code est composé de trois principaux packages :

- atoudeft (et ses sous-package) : implémente le noyau du serveur qui s'occupe de la connexion et des échanges avec les clients.
- demo : contient principalement la classe Main pour le lancement du serveur.
- gestionnaireLivraisons : implémente la gestion effective des livraisons.
- tests : ce package va contenir les classes de tests que vous allez implémenter.

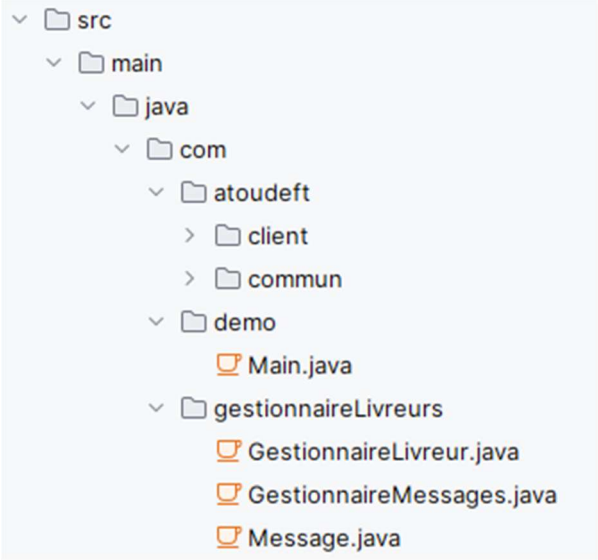
Pour la réalisation de ce travail, vous allez implémenter ou compléter des classes du package **gestionnaireLivraisons**. Vous n'aurez pas besoin d'apporter des modifications

dans le package atoudeft (et ses sous-packages). En fait, **vous ne devez faire aucune modification** dans ces packages.

LE CLIENT

Chaque livreur va lancer son application client (projet miniClient). Cette application va, dans une boucle, simplement attendre la saisie d'une commande et la traiter. Le traitement consiste principalement à envoyer la commande au serveur. Cependant, l'envoie ne se fait que si le livreur s'est connecté au serveur. Par conséquent, la première commande du livreur, une fois l'application client lancée, est de se connecter. Il s'agit de la commande : ***READY***

La figure suivante montre la structure du code de l'application client :



Le code est composé de trois principaux packages :

- atoudeft (et ses sous-package) : implémente le noyau du client qui s'occupe de la connexion et les échanges avec le serveur.
- demo : contient la classe Main pour le lancement du client.
- gestionnaireLivreurs : implémente la gestion des livreurs.

L'application client est complète. Vous ne devez pas y toucher.

Démarrage des applications

Il s'agit de l'application serveur et l'application client qui s'exécutent en mode console. L'interaction avec l'utilisateur se fait par commandes que ça soit du côté serveur ou du côté client.

Démarrage du serveur

L'application serveur (miniServer) doit être démarrée en premier (la classe Main) pour qu'il soit à l'écoute avant de démarrer l'application client. Le serveur démarre et écoute sur le port 8888 l'arrivée de demandes de connexions.

Le port 8888 est le port d'écoute par défaut et est déclaré dans l'interface Config. Pour changer de port, il faut modifier cette valeur dans les interfaces Config à la fois dans l'application serveur et l'application cliente.

Le serveur se tient aussi à l'écoute des commandes que les clients connectés peuvent envoyer. Un message est affiché sur la console (du serveur) à la réception des commandes.

L'utilisateur peut utiliser les commandes suivantes :

- EXIT : le serveur s'arrête
- HELP : affichage de l'aide
- LIST : affiche la liste des livraisons à effectuer
- STATS : affiche des statistiques

Démarrage du client

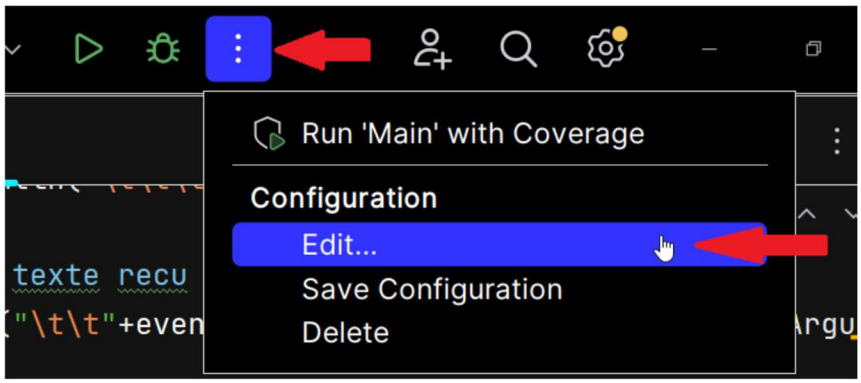
Une fois le serveur lancé, on peut exécuter l'application client permettant au livreur d'interagir avec le serveur. Sur la console de l'application (client), le livreur peut utiliser les commandes suivantes :

- HELP : afficher cette liste de commandes.
- READY : se connecter au serveur.
- REGISTER <id livreur> <type livreur> <nom livreur> : s'enregistrer auprès du serveur comme nouveau livreur.
- ID <id livreur> : s'identifier auprès du serveur.
- GET : demander des livraisons à effectuer.
- DELIVERED <id livraison> : signifier que la livraison <id livraison> a été livrée avec succès.

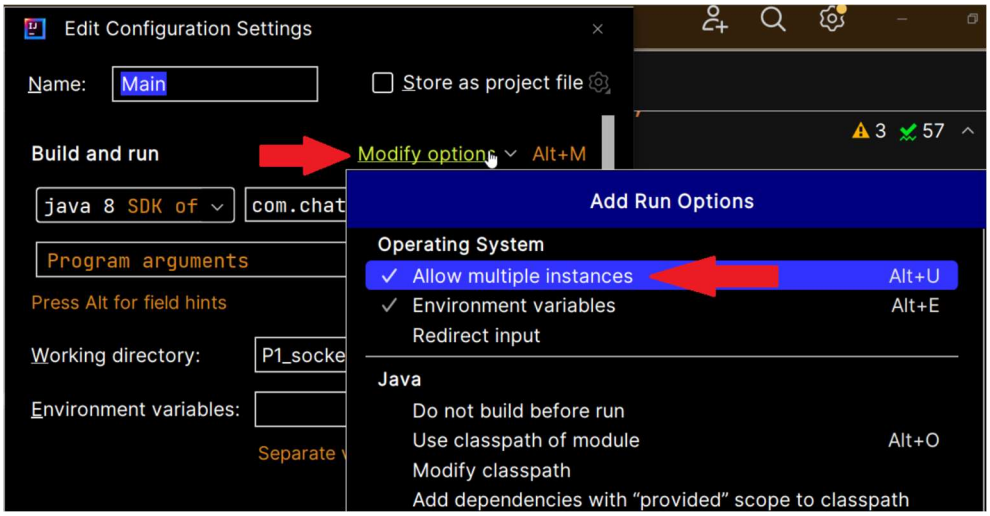
- FAILED <id livraison> : signifier que la livraison <id livraison> n'a pas pu être livrée.
- INFO [<id livraison>] : demander des informations sur les livraisons en cours.
- INCOME : demander le revenu des livraisons effectuées durant cette session.
- SEND {<id livreur>|*} : envoyer un message à un livreur ou à tous les livreurs.
- EXIT : quitter l'application (client)

Noter que pour chaque commande envoyée au serveur, le client recevra une réponse qui sera affichée sur la console du client.

Pour bien tester votre code et avoir une simulation plus réaliste, on aura besoin de lancer plusieurs clients (c'est-à-dire, exécuter plusieurs fois l'application client). Pour pouvoir le faire sous IntelliJ, il faut éditer la configuration d'exécution :



Ensuite, dans la fenêtre qui apparaît, dérouler l'item ***Modify options*** et cocher l'option ***Allow multiple instances***.



Gestion des livraisons – travail à réaliser

Le travail à réaliser est divisé en deux parties :

1) Implémentation de la partie relative à la gestion des livraisons
2) Implémentation de la partie consacrée aux traitements des événements reçus par le serveur

Pour la réalisation de chaque partie, vous avez une série de questions auxquelles, il faut répondre.

Noter que le présent document concerne la partie 1 du TP. Il est nécessaire de compléter l'implémentation de la partie 1 avec de faire celle de la partie 2.

Vous recevrez les questions de la partie 2 dans un autre document, très prochainement.

Partie I : la gestion des livraisons

Question 1.1 :

Implémenter l'énumération Priorite ayant les labels : *NORMALE* et *URGENTE*

Question 1.2 :

Implémenter l'énumération Statut ayant les labels : *EN_ATTENTE*, *EN_COURS*, *LIVREE* et *ECHOUEE*

Question 1.3 :

Pour représenter les livraisons dans notre application, nous allons implémenter la classe Livraison (dans le package gestionnaireLivraisons) ayant les attributs suivants :

- *id* : un identifiant entier représentant un numéro unique de la livraison
- *priorite* de type Priorite (énumération) : indique le niveau de priorité de la livraison
- *tentative* de type int : indique le numéro de la tentative. Cet attribut doit être initialisé à 0 à la création de l'objet Livraison
- *lot* de type int : indique le numéro de lot auquel appartient la livraison
- *statut* de type Statut (énumération) : pour indiquer le statut de la livraison. Il doit être initialisé à *EN_ATTENTE*

Question 1.4 :

Dans la classe Livraison :

- Définir la constante nommée *MAX_TENTATIVES* valant 3
- Implémenter la méthode *nouvelleTentative*() qui incrémente le nombre de tentatives, sans dépasser le nombre maximum défini par la constante MAX_TENTATIVES, et retourne true. Si le nombre de tentative a déjà atteint MAX_TENTATIVES, la méthode retourne false.

Question 1.5 :

Dans la classe Livraison, implémenter une méthode statique nommée *prochainID*() qui retourne un numéro séquentiel à chaque appel, **commençant à 1**. Cette méthode servira à initialiser l'attribut **id** de Livraison. Indication : déclarer un attribut statique dans la classe Livraison qui gardera le numéro séquentiel.

Question 1.6 :

Pour la classe Livraison, implémenter un constructeur pour initialiser les 5 attributs de la classe Livraison. Le constructeur prendra deux paramètres : le niveau de priorité de type Priorite et le numéro du lot.

Question 1.7 :

Redéfinir la méthode *toString()* permettant l'affichage des 5 attributs de la classe Livraison.

Question 1.8 :

Implémenter le test d'inégalité, selon l'ordre naturel, basé sur les attributs : *lot*, *priorite* (*NORMALE* et *URGENTE*) et *tentative*. Cela permettra d'avoir les objets Livraison triés dans l'ordre de livraison dans la **file à priorité**. L'ordre sur les livraisons est défini comme suit :

- Les livraisons d'un lot donné doivent être livrées avant celles ayant un numéro de lot plus grand. Par exemple, les livraisons du lot 2 doivent être livrées avant celles du lot 3.
- Pour les livraisons du même lot, celles avec la priorité *URGENT* doivent être livrées avant celles ayant la priorité *NORMALE*
- Pour les livraisons du même lot et de même priorité, la livraison ayant le numéro de tentative plus élevé doit être livrée avant celles ayant le numéro de tentative plus bas

Ainsi, les livraisons d'un même lot, d'une même priorité et de même numéro de tentatives sont considérées comme « égales » selon ces critères. Elles seront placées dans l'ordre dans lequel elles ont été ajoutées dans la file à priorité.

Rappelons que pour l'ordre naturel, cela consiste à redéfinir la méthode *compareTo()* (*obj_1.compareTo(obj_2)*) qui retourne -1, si le premier objet (*obj_1*) est plus petit que le deuxième objet (*obj_2*), retourne 0 si les deux objets sont égaux et retourne 1 si le premier objet (*obj_1*) est plus grand que le deuxième objet (*obj_2*). Consulter les notes de cours.

Question 1.9 :

Pour stocker les livraisons selon l'ordre dans lesquelles elles doivent être livrées, nous allons implémenter la classe nommée FilePrioriteLivraisons qui encapsule un objet de la classe *java.util.PriorityQueue*. Cette dernière permet d'insérer un objet à la bonne position dans la file, en mettant en application le test d'inégalité de l'ordre naturel. Concrètement, la position d'insertion est déterminée en comparant les objets par invocation de la méthode *compareTo()* que vous avez implémentée dans la classe Livraison.

Votre tâche dans cette question est de compléter l'implémentation de la classe FilePrioriteLivraisons.

Question 1.10 :

Implémenter la classe TestFilePrioriteLivraisons qui va tester chacune des méthodes de la classe FilePrioriteLivraisons. Les méthodes sont présentes dans la classe mais leur implémentation reste à faire.

Question 1.11 :

Compléter la méthode statique *populateFileLivraisons*() de la classe LivraisonFactory, dont la signature est la suivante :

```
public static FilePrioriteLivraisons populateFileLivraisons()
```

Cette méthode va créer des objets Livraison, en utilisant les données sous la forme de chaîne de caractères dans le tableau statique dataLivraisons. En effet. Chaque élément du tableau est une chaîne de caractères contenant le numéro de lot et la priorité (NORMALE ou URGENTE) d'une livraison (Exemple : "1:NORMALE"). Il faut décomposer cette chaîne de caractères pour extraite les données qui serviront à la création de l'objet Livraison. Les objets Livraison ainsi créés doivent être ajoutés dans la file à priorité (Objet FilePrioriteLivraisons).

Cette méthode retourne l'objet FilePrioriteLivraisons.

Question 1.12 :

Implémenter la classe ListeLivraisons pour stocker une liste d'objets Livraison, en utilisant un objet ArrayList comme structure de données interne. Cette classe doit implémenter l'interface IListeLivraisons.

La classe doit également implémenter l'interface Iterable afin de redéfinir la méthode iterator() (`public ListIterator<Livraison> iterator()`). Cette méthode retourne un itérateur des objets Livraison de la liste.

Question 1.13 :

Implémenter l'exception nommée ListeChaineeException. Le constructeur prend un attribut message de type String. Il servira à initialiser l'attribut « message » de la classe mère. Cette Exception sera utilisée dans la classe ListeChaineeLivreurs pour les situations d'erreurs.

Question 1.14 :

Implémenter la classe ListeChaineeLivreurs pour stocker une liste de livreurs en utilisant une liste à chainage simple. Elle doit implémenter l'interface IListeChaineeLivreurs.

Question 1.15 :

Implémentation des classes pour représenter les livreurs. Il s'agit de la classe abstraite Livreur et de ses sous-classes LivreurVelo, LivreurAuto et LivreurCamion.

Les attributs de la classe Livreur :

- *Id* : un entier représentant l'identifiant du livreur
- *nom* : une chaîne de caractères pour stocker le nom du livreur
- *livraisonsEnCours* : un objet de type ListeLivraisons pour stocker les livraisons en cours du livreur, c'est-à-dire les livraisons qui ont été attribuées au livreur pour les effectuer
- *livraisonsEffectuées* : un objet de type ListeLivraisons pour stocker les livraisons effectuées avec succès par le livreur

Dans cette classe, implémenter le constructeur et les méthodes publiques suivantes :

- Un constructeur pour l'initialisation des attributs. Il prend l'identifiant et le nom du livreur comme paramètres
- La méthode `void ajouterLivraisonEnCours(Livraison livraison)`. Cette méthode ajoute un objet Livraison à la liste des livraisons en cours du Livreur
- La méthode `void ajouterLivraisonEffectuee(Livraison livraison)`. Cette méthode ajoute l'objet Livraison dans la liste des livraisons déjà effec-

tuées par le livreur

- La méthode `Livraison supprimerLivraisonEnCours(int idLivraison)`. Cette méthode retire la livraison de la liste des livraisons en cours, à partir de l'identifiant. La méthode retourne l'objet supprimé, sinon retourne null si la livraison n'existe pas.
- La méthode `boolean aDesLivraisonsEnCours()`. Cette méthode retourne true si le livreur a des livraisons en cours, retourne false sinon
- La méthode `IListeLivraisons supprimerToutesLesLivraisons()`. Cette méthode vide la liste des livraisons en cours. Les objets supprimés sont placés dans une nouvelle liste (de type ListeLivraisons) qui sera retournée comme résultat
- La méthode `Livraison rechercherLivraisonEnCours(int idLivraison)`. Cette méthode retourne l'objet Livraison de la liste des livraisons en cours, à partir de son identifiant. Elle retourne null si l'objet n'existe pas.

En outre, vous aurez besoin d'implémenter les méthodes suivantes :

- La méthode `int capaciteLivraison()`. Cette méthode retourne le nombre de livraisons qu'un livreur peut livrer selon le moyen de livraison utilisé. En effet :
  - La capacité d'un livreur par vélo est de **deux** livraisons
  - La capacité d'un livreur par voiture est de **cinq** livraisons
  - La capacité d'un livreur par camion est de **huit** livraisons
- La méthode `double calculerRevenu()`. Cette méthode calcule le revenu d'un livreur selon les formules suivantes :
  - Le revenu d'un livreur à vélo est de 5.0$ par livraison
  - Le revenu d'un livreur en voiture à vélo est de 7.5$ par livraison
  - Le revenu d'un livreur en camion est de 10.0$ par livraison
- La méthode `String toString()`. Il faut redéfinir cette méthode. Elle doit retourner une chaîne de caractères incluant la nature du livreur (à vélo, en voiture ou par camion), l'identifiant et le nom du livreur

Note : les classes LivreurVelo, LivreurAuto et LivreurCamion n'ont pas d'attributs d'instances.

Partie 2 : Traitement des commandes

Dans la partie 2 du TP, vous allez implémenter le traitement des commandes reçues par le serveur. En effet, les livreurs envoient des commandes au serveur par le biais de l'application client (MiniClient). La communication entre les 2 applications utilise le mécanisme de sockets, implémenté dans la librairie standard de Java.

Les événements

L'élément d'interaction entre 2 entités qui communiquent, est l'événement. Dans notre contexte, l'événement correspond à la réception d'une commande par le serveur ou la réception d'une réponse par le client. Pour représenter un tel événement, on a défini la classe Evenement (package *com.atoudeft.commun.evenement*) :

```
public class Evenement {
  private final Object source;
  private final String type, argument;

  /**
   * Construit un événement.
    * @param source   La source de l'événement
   * @param type     Le type de l'événement
   * @param argument Chaîne de caractères contenant les arguments de l'événement
   */
  public Evenement(Object source, String type, String argument) {
    this.source = source;
    this.type = type;
    this.argument = argument;
  }
  /**
   * Retourne la source de l'événement
   * @return Object source de l'événement
   */
  public Object getSource() {
    return source;
  }
  /**
   * Retourne le type de l'événement
   * @return String type de l'événement
   */
  public String getType() {
    return type;
  }
  /**
   * Retourne l'argument de l'événement
   * @return String argument de l'événement
   */
  public String getArgument() {
    return argument;
  }
}
```

Description des attributs :

- source : indique l'expéditeur (Instance de la classe Connexion – voir la suite)
- type : indique le nom de la commande
- arguments : c'est une chaîne de caractères composée des arguments de la commande, séparés par un espace

Les arguments

Certaines commandes ont un ou plusieurs arguments qui sont stockés dans l'objet Evenement. La méthode *getArgument()* de cette classe retourne les arguments en une seule chaine de caractères.

Afin de séparer les arguments contenus dans la chaine de caractères, on peut utiliser la classe Arguments du package *com.atoudeft.commun.evenement*, qui fait l'extraction des arguments en invoquant les méthodes :

- extraireArgumentSuivant() : retourne le prochain argument à chaque appel. Retourne null s'il ne reste plus d'arguments
- lire() : retourne le reste des arguments en une seule chaine de caractères

Exemple :

```
Arguments args = new Arguments(evenement);
String unArgument = args.extraireArgumentSuivant();
String leResteDesArguments = args.lire();
```

Les connexions

Un autre élément important dans une application client-serveur est la connexion. Un client se connecte au serveur pour qu'il soit « reconnu » par lui et de pouvoir entamer leurs échanges par la suite. On a défini la classe Connexion (package *com.atoudeft.commun.net*). Ainsi, à chaque connexion d'un livreur, un objet Connexion est créé au niveau du serveur.

Les Livreurs enregistrés

L'application serveur dispose d'une liste de livreurs qui se sont enregistrés à un moment donné. Les informations sur les livreurs enregistrés sont stockées dans un fichier nommé *livreurs.txt*. Ainsi, au démarrage, le serveur va lire ce fichier pour créer des objets correspondants aux livreurs et les stocker dans la liste chainée que vous avez implémentée. Par la suite, un livreur enregistré peut s'authentifier.

Le fichier *livreurs.txt* prédéfinie dans l'application est le suivant :

```
101 VELO Pierre
102 CAMION Paul
103 VOITURE Jacques
```

- 1ère donnée indique l'identifiant du livreur
- 2ème donnée indique le mode de livraison
- 3ème donnée indique le nom du livreur

Un nouveau livreur peut également s'enregistrer en utilisant la commande REGISTER (cette commande sera présentée avec plus de détail dans la suite du document). Il devra se connecter en premier puis s'enregistrer.

À la fin, l'application serveur enregistre la liste des livreurs dans le fichier *livreurs.txt* avant de quitter.

Démarrage du serveur

Au lancement de l'application MiniServeur, la méthode *main()* fait, entre autres, la création des objets Serveur et GestionnaireLivraisons, puis invoque la méthode *demarrer()* pour lancer le serveur qui, après une étape d'initialisation, va se mettre à l'écoute des commandes envoyées par les livreurs. À la réception d'une commande, la méthode *lire()* de la classe Serveur, récupère cette commande, crée l'objet Evenement puis invoque la méthode *traiter()* de la classe GestionnaireLivraisons.

```
public synchronized void lire() {
  String[] t;
  Evenement evenement;
  Connexion connexionToRemove = null;

  for (Connexion cnx : connectes) {
    String texte = cnx.getAvailableText();
    if (texte == null) {
      connexionToRemove = cnx;
    } else if (!texte.isEmpty()) {
      t = EvenementUtil.extraireInfosEvenement(texte);
      evenement = new Evenement(cnx, t[0], t[1]);
      gestionnaireEvenementServeur.traiter(evenement);
    }
  }
  this.connectes.remove(connexionToRemove);
}
```

La classe GestionnaireLivraisons

GestionnaireLivraisons est la classe principale qui implémente la gestion des livraisons. Les principaux attributs de cette classe sont les structures de données suivantes :

- final private IListeChaineeLivreurs livreursEnregistres : liste chaînée pour contenir les livreurs enregistrés. Elle est initialisée à la lecture du fichier *livreurs.txt*. Lorsqu'un nouveau livreur s'enregistre, il sera ajouté à cette liste
- final private Hashtable<Connexion, Livreur> livreursAuthentifies : une table de hachage pour contenir les livreurs qui se sont identifiés. À chaque fois qu'un livreur s'identifie, il est ajouté à la table, et lorsqu'il quitte, il devrait en être retiré
- final private FilePrioriteLivraisons livraisonsAEffectuer : file à priorité qui contient l'ensemble des livraisons qui doivent être effectuées. À l'initialisation, des livraisons prédéfinies sont stockées dans cette liste à l'invocation de la méthode statique *populateFileLivraisons()*. Les livraisons qui doivent être attribuées à un livreur seront prises de cette liste. On y remettra également les livraisons non effectuées ou celles à la suite d'un échec de livraison, afin de procéder à une autre tentative

- final private IListeLivraisons livraisonsEchouees : Liste pour stocker les livraisons qui ont échouées après trois tentatives

Le constructeur de classe GestionnaireLivraisons fait principalement la création et l'initialisation des structures de données.

Une méthode importante de cette classe est la méthode *traiter()* qui traite les commandes reçues par le serveur :

```
public void traiter(Evenement evenement) {
  Object source = evenement.getSource();

  if (source instanceof Connexion) {
    Connexion cnx = (Connexion) source;

    String reponse = "";
    switch (evenement.getType().toUpperCase()) {
      case "EXIT": // Le client se déconnecte.
        reponse = this.traiterEXIT(evenement);
        break;
      case "REGISTER": // Le client s'enregistre comme nouveau livreur.
        reponse = this.traiterREGISTER(evenement);
        break;
      case "ID": // Le client s'identifie.
        reponse = this.traiterID(evenement);
        break;
      case "GET": //  Le client a demandé des livraisons à effectuer.
        reponse = this.traiterGET(evenement);
        break;
      case "DELIVERED": // Le client informe qu'une livraison a été effectuée.
        reponse = this.traiterDELIVERED(evenement);
        break;
      case "FAILED": // Le client informe qu'une livraison n'a pas pu être effectuée.
        reponse = this.traiterFAILED(evenement);
        break;
      case "INCOME":
        reponse = this.traiterINCOME(evenement);
        break;
      case "INFO":
        reponse = this.traiterINFO(evenement);
        break;
      case "SEND":
        reponse = this.traiterSEND(evenement);
        break;
      default: // La commande envoyée par le client n'a pas été reconnue.
        this.traiterCOMMAND_ERROR();
    }
    cnx.envoyer(reponse);
  }
}
```

La méthode *traiter()* invoque la méthode qui correspondant à la commande reçue puis envoie une réponse au livreur(*cnx.envoyer(reponse)*). **Ainsi, à chaque commande, le serveur <u>retourne une réponse</u> au livreur pour confirmer le traitement de sa com-**

mande, ou pour indiquer une erreur. <u>Consulter la liste des réponses ci-bas, pour constituer la réponse à une commande</u>.

Pour plusieurs commandes, le traitement consiste, entre autres, à <u>mettre à jour une ou plusieurs structures de données</u> (ajout, suppression, etc.).

La liste suivante indique les différentes réponses du serveur au livreur lorsque la commande est traitée avec succès :

- "CONNECTED" : Le serveur a accepté la connexion du livreur.
- "REGISTERED <id livreur> <nom livreur>" : Le serveur a enregistré le nouveau livreur
- "AUTHORIZED <id livreur> <nom livreur>" : Le serveur a bien identifié le livreur.
- "DELIVERIES <nb livraisons> {<id livraison> <lot livraison> <priorité> <numéro de tentative>} +" : Le serveur envoie des livraisons au livreur en réponse à la commande GET
- "DELIVERIES_INFO <nb livraisons> {<id livraison> <lot livraison> <priorité> <numéro de tentative>} +" : Le serveur envoie les livraisons en cours pour ce livreur en réponse à la command INFO
- "DELIVERED_OK <id livraison>" : Le serveur confirme qu'une livraison a bien été livrée par le livreur
- "FAILED_CONTINUE <id livraison>" : Le serveur confirme que ce livreur n'a pas pu livrer une livraison et qu'elle a été remise dans les livraisons à effectuer
- "FAILED_ABORT <id livraison>" : Le serveur confirme que ce livreur n'a pas pu livrer une livraison et qu'elle a été placée dans les livraisons non livrées
- "EMPTY" : Le serveur informe le livreur qu'il n'y a plus de livraisons à effectuer en réponse à la commande GET
- "REVENU <montant> <nombre de livraisons>" : Le serveur retourne le montant gagné et le nombre de livraisons effectuées à date par ce livreur
- "ACK" : Le serveur accuse réception d'un message après une commande SEND par ce livreur. Voir la partie 3
- "MSG <id livreur> <message>" : Le serveur envoie un message issu d'un autre livreur. Voir la partie 3
- "END" : Le serveur a déconnecté le client. Ce message est envoyé aux livreurs avant l'arrêt du serveur ou lorsqu'un livreur se déconnecte

Dans les situations d'erreurs, le serveur envoie l'un des messages suivants :

- "AUTHENTICATION_ERROR" : Le serveur n'a pas pu identifier ce livreur
- "ALREADY_AUTHENTIFIED_ERROR" : Le serveur informe que ce livreur est déjà identifié

- "TOO_MANY_CONNECTIONS_ERROR" : Le serveur informe le livreur qu'il est déjà identifié sur un autre client. C'est lorsque le livreur se connecte à partir de plusieurs applications MiniClient
- "COMMAND_ERROR" : Le serveur n'a pas reconnu la commande envoyée par ce livreur
- "BAD_ARGUMENT_ERROR" : La commande envoyée par le livreur n'avait pas les bons paramètres
- "NO_DELIVERY_ERROR" : Le serveur informe le client qu'il n'a pas de livraisons en cours
- "BAD_DELIVERY_ERROR" : Le serveur informe ce livreur que la livraison spécifiée dans la précédente commande n'est pas une livraison en cours pour ce livreur
- "UNIMPLEMENTED_ERROR" : La précédente commande envoyée par ce livreur existe mais elle n'est pas encore implémentée par le serveur.
- "ALREADY_REGISTERED_ERROR" : Le serveur informe le livreur qu'il est déjà enregistré après avoir reçu une commande REGISTER

Précision :

Il faut bien interpréter une description comme :

"DELIVERIES <nb livraisons> {<id livraison> <lot livraison> <priorité> <numéro de tentative>} +"

- Le rôle des symboles < et > est de délimiter un paramètre. Il ne fera pas partie de la réponse
- Le rôle des symboles { et } est de regrouper un ensemble de paramètres
- Le rôle du symbole + indique que les paramètres regroupés entre { et } sont répétés une ou plusieurs fois

Pour l'exemple DELIVERIES, on pourrait avoir la réponse suivante :

DELIVERIES 2 5 1 URGENTE 1 12 1 NORMALE 1

Implémentation du traitement des commandes

Question 2.1 :

Dans la classe GestionnaireLivraisons, complétez l'implémentation de la méthode *lireFfichierLivreurs*(). Cette méthode lit le fichier des livreurs enregistrés (*livreurs.txt*), crée les objets correspondants puis les ajoute dans la liste chaînée nommée livreursEnregistres.

Question 2.2 : traitement de la commande ID

Après qu'un livreur se connecte au serveur, il devrait s'authentifier en indiquant son identifiant, et cela avec la commande ID (exemple : ID 102). Cette commande est traitée par la méthode *traiterID(Evenement evenement)* qu'il faudra compléter.

Les principales opérations de cette méthode sont :

- Retrouver l'objet livreur correspondant à l'identifiant donné dans la commande
- Mettre cet objet dans la table livreursAuthentifies avec comme clé, l'objet Connexion correspondant au livreur.

Attention : cette méthode doit s'assurer que le livreur est déjà enregistré, qu'il ne s'est pas déjà authentifié avec la même connexion ou avec une connexion différente.

Question 2.3 : traitement de la commande GET

Livreur connecté et identifié peut demander au serveur de lui attribuer des livraisons à effectuer en envoyant la commande GET. Le serveur détermine le nombre de livraisons à lui attribuer, prend ce nombre de livraisons puis envoie les informations de ces livraisons au livreur.

Pour cette tâche, compléter l'implémentation de la méthode *traiterGET(Evenement evenement).* Parmi les opérations de cette méthode :

- S'assurer que le livreur s'est authentifié
- Déterminer le nombre de livraisons à attribuer en fonction de la capacite du livreur et le nombre de livraisons déjà attribuées au livreur, mais qu'il n'a pas encore livrées
- Composer la chaine de caractères qui constitue la réponse au livreur

Question 2.4 : traitement de la commande INFO

Il y a 2 options pour la commande INFO :

- La commande INFO sans paramètres : permet au livreur de revoir la liste des livraisons qui lui sont attribuées
- La commande INFO <id_livraison> : permet au livreur d'avoir les informations d'une livraison identifiée par le paramètre *id_livraison*

Pour le traitement de la commande INFO, compléter la méthode *traiterINFO(Evenement evenement)* qui doit traiter les 2 options de la commande, Parmi les opérations de cette méthode :

- Vérifier que le livreur s'est déjà authentifié
- Pour l'option sans paramètres, retrouver la liste des livraisons qui ont été attribuées au livreur, construire la réponse à envoyer au livreur
- Pour l'option avec paramètre, retrouver la livraison identifiée par l'ID donné en paramètre, préparer la réponse à envoyer au livreur

Attention : il faut prendre en compte les situations d'erreurs comme id de la livraison qui n'existe pas, etc.

Question 2.5 : traitement de la commande DELIVERED

Lorsque le livreur effectue une livraison avec succès, il envoie la commande DELIVERED <id_livraison> au serveur. Pour le traitement de cette commande, compléter la méthode *traiterDELIVERED(Evenement evenement)*. Parmi les opérations de cette méthode :

- Vérifier que le livreur s'est déjà authentifié
- S'assurer que l'identifiant de la livraison est celui d'une livraison en cours attribuée au livreur

Question 2.6 : traitement de la commande FAILED

Lorsque le livreur échoue à effectuer une livraison, il envoie la commande FAILED <id_livraison> au serveur. Pour le traitement de cette commande, compléter la méthode *traiterFAILED(Evenement evenement).* Parmi les opérations de cette méthode :

- Vérifier que le livreur s'est déjà authentifié
- S'assurer que l'identifiant de la livraison est celui d'une livraison en cours attribuée au livreur

- Remettre la livraison dans la file des livraisons à effectuer en mettant à jour le nombre de tentative, sachant qu'on fait 3 tentatives au maximum. S'il est atteint, la livraison sera stockée dans la liste des livraisons échouées

Question 2.7 : traitement de la commande INCOME

Un livreur peut consulter le revenu réalisé par son activité en envoyant la commande INCOME au serveur. Pour le traitement de cette commande, compléter la méthode *traiterINCOME(Evenement evenement)*. Parmi les opérations de cette méthode :

- Vérifier que le livreur s'est déjà authentifié
- Calculer le revenu du livreur selon les livraisons réalisées

Question 2.8 : traitement de la commande EXIT

Le livreur a la possibilité de se déconnecter du serveur en envoyant la commande EXIT. Pour le traitement de cette commande, compléter la méthode *traiterEXIT(Evenement evenement)*. Parmi les opérations de cette méthode :

- Retrouve l'objet Livreur correspondant au livreur ayant envoyé la commande
- Retrouver les livraisons qui ont été attribuées mais pas encore traitées pour les remettre dans la file des livraisons à effectuer. Ici, on ne change pas le numéro de tentative de la livraison

Question 2.9 : traitement de la commande REGISTER

Après la connexion, un nouveau livreur peut s'enregistrer au niveau du serveur en envoyant la commande REGISTER <id_livreur > <mode_livraison> <nom_livreur>.

Pour le traitement de cette commande, compléter la méthode *traiterREGISTRER(Evenement evenement)*. Parmi les opérations de cette méthode :

- Récupérer les paramètres de la commande
- S'assurer que les paramètres sont valides
- S'assurer que l'identifiant n'est pas déjà utilisé par un autre livreur
- Créer l'objet livreur selon le mode de livraison et le stocker dans la liste des livreurs enregistrés

<mode_livraison> doit être : VELO, CAMION ou VOITURE

Partie III : messagerie entre livreurs

Une autre fonctionnalité de l'application client-serveur pour la gestion des livraisons, est la messagerie entre les livreurs. Ainsi, un livreur a la possibilité d'envoyer un message à un livreur en particulier ou à l'ensemble des livreurs authentifiés (diffusion) grâce à la commande SEND :

- SEND <id livreur> <texte du message> : envoie un message à un livreur en particulier
- SEND * <texte du message> : envoie un message à tous les autres livreurs. Noter que * c'est pour indiquer que c'est une diffusion

Lorsqu'un livreur envoie un message, c'est le serveur qui le récupère pour le transmettre à un livreur en particulier ou à l'ensemble des autres livreurs authentifiés. Notez que le message reçu au niveau du Serveur contient un identifiant unique (*identifiant_msg*) et un contenu.

Après avoir transmis le message, le serveur répond au livreur expéditeur par la commande ACK <identifiant_msg>

Cependant, à cause de l'instabilité de la communication, des messages peuvent être perdus et ne jamais être reçus par le serveur. Pour pallier ce problème, le client envoie de manière répéter le même message (tous les 500 millisecondes) jusqu'à la réception de la commande ACK de la part du Serveur, prouvant que le message a bien été reçu.

Par conséquent, le serveur peut recevoir plusieurs fois un même message d'un livreur, la première fois, le message est traité (c'est-à-dire transmis au(x) destinataire(s)), pour les autres fois, le message est simplement ignoré.

Question 3.1 : traitement de la commande SEND

Pour le traitement de la commande SEND, compléter la méthode *traiterSEND(Evenement evenement)*. Parmi les opérations de cette méthode :

- S'assurer que le livreur expéditeur s'est bien authentifié
- Détermine si ce même message (selon son identifiant) a déjà été reçu et traité, dans ce cas, le message est ignoré
- Si le message est reçu pour la première fois, il est transmis au destinataire ou diffusé à tous les autres livreurs authentifiés
- Retourner la commande ACK <identifiant_msg> à l'expéditeur

Attention : il faut traiter les cas d'erreur comme destinataire non authentifié, paramètres non valides, etc.

Indications et remises

- Créer un dépôt github privé pour votre travail et m'envoyer une invitation (utilisateur elhachemiETS)
- Remise de votre projet zippé dans Moodle. À remettre uniquement le projet MiniServeur
- Date limite de remise est le **18 mars 2026**, 23h59, au plus tard

Bon travail!